

AWS GenAI 課程 · 單元 D02

DevOps 核心結構 與 GitHub 基礎

盧建霖 · 2026-09-17

Agenda

- 01 文化與自動化
- 02 Git 的原理與指令
- 03 GitHub 協作與規矩
- 04 AI 時代的責任與實作

TODAY

八個知識點，一小時

每段結束有三題隨堂測驗

TAKEAWAY

工具固定流程，
文化決定流程

DevOps 是文化，不是工具

要回答的問題：為什麼公司買了一堆自動化工具，部署還是一樣痛苦？

- 一、開發與維運為什麼會吵架
- 二、**CALMS** 五個面向一次看
- 三、為什麼文化要先於工具

開發與維運為什麼會吵架

比較項目	開發 (Dev)	維運 (Ops)
被考核什麼	功能出得快	系統不要掛
最想要	常常改	盡量別動
出事時的第一句	「在我電腦上可以跑」	「你們又改了什麼」

兩邊的目標天生衝突，不是誰態度不好。

CALMS: 五個面向, 一次看

DevOps Development 加 Operations 的合成詞, 讓寫程式的人與顧系統的人用同一套流程、對同一個結果負責。

CALMS 五個字的縮寫, 用來評估一個組織 DevOps 做到哪; 由 Jez Humble 提出。

C

Culture
文化

開發與維運坐同一張桌子,
對同一個結果負責

沒做到: 工具再多也是互丟

A

Automation
自動化

建置、測試、部署交給機器重
複做

沒做到: 靠人記步驟, 一定漏

L

Lean
精實

小批量、快回饋, 不囤一大包
才交付

沒做到: 出事找不到是哪一行

M

Measurement
量測

部署頻率、修復時間這類數
字看得到

沒做到: 「感覺變好了」無法改
進

S

Sharing
分享

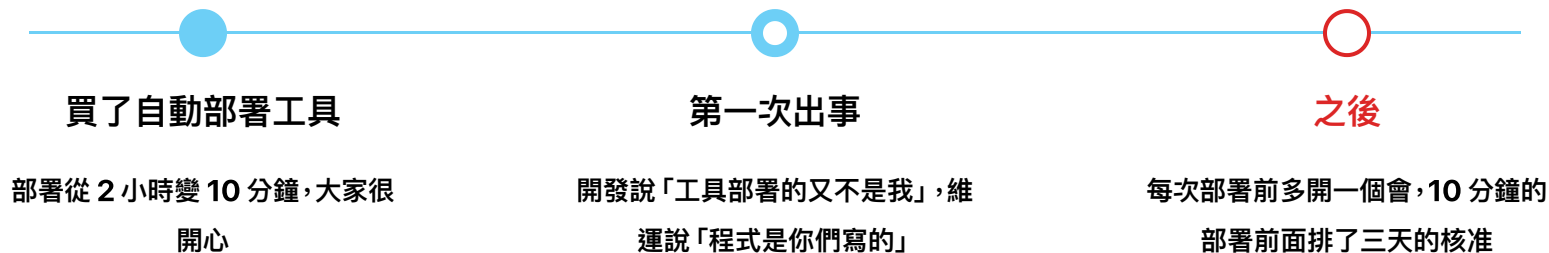
出事的復盤公開, 知識不卡
在一個人身上

沒做到: 同一個坑每人踩一次

Culture 上色是因為: 其他四個都建在它上面。

一個團隊的故事：自動化之後發生什麼

講師的綜合觀察，不是特定公司



沒有 **Culture** 的 **Automation**，只是更快地把問題丟過牆。

隨堂測驗：DevOps 文化與 CALMS

Q1 一個團隊部署全自動，但每次出事開發與維運都互相推。CALMS 裡最缺哪一個？

A Automation

B Culture

C Measurement

D Lean

Q2 「三個月發一次大版本」違反哪一個面向？

A Sharing

B Culture

C Lean

D Automation

Q3 下列哪一句最接近本課對 DevOps 的定義？

A 一個負責部署的職位

B 一套工具，例如 Docker 加 Kubernetes

C 開發與維運用同一套流程、對同一個結果負責

D 把維運外包出去

想 30 秒，翻下一頁對答案。

解答: DevOps 文化與 CALMS

Q1 **B Culture**

自動化已經有了,缺的是對同一個結果負責的文化。

Q2 **C Lean**

精實要的是小批量、快回饋;三個月一包就是反例。

Q3 **C 同一套流程、同一個結果**

A 與 B 是最常見的誤解;課綱明說 DevOps 是組織與流程的問題。

一句話帶走: **DevOps** 不解決工具問題,它解決人的問題。

CI、CD、CD：三個字，兩件事

要回答的問題：為什麼「常常提交」反而比較不痛？

- 一、一條 **pipeline** 長什麼樣
- 二、三個名詞，差在自動化做到哪裡
- 三、為什麼提交頻率決定整合痛苦程度

一條 pipeline 長什麼樣

pipeline (管線, 一連串自動執行的階段, 前一段過了才進下一段)



每一段失敗就停, 後面不跑, 這就是「擋」。



三個名詞，差在自動化做到哪裡

名詞	白話	範圍到哪
持續整合 Continuous Integration, CI	開發者常常把改動合併進中央版本庫，合併後自動建置與測試	到測試為止
持續交付 Continuous Delivery, CD	在 CI 之後，把每次改動自動準備到可以發布的狀態，部署到測試環境	到正式環境門口， 上線前有人核准
持續部署 Continuous Deployment	連上正式環境那一下都自動	全程沒有人工核准

差別只有一件事：**有沒有人工核准上正式環境**。很多團隊做到的其實是持續交付。

為什麼提交頻率決定整合痛苦程度

✗ 兩個人各改兩週才合併

幾十個檔案互相踩到，衝突一大片

找錯的範圍：兩週的所有改動

✓ 每天合併一次

每次只有幾個檔案，衝突當天解

找錯的範圍：今天的改動

痛苦不是來自整合，是來自**累積**。

擋在今天，不要擋在兩週後。

隨堂測驗：CI、CD、CD

Q1 團隊每次改動都自動建置、測試、部署到測試環境，上正式環境要主管點一下。這是哪一種？

A 持續整合

B 持續交付

C 持續部署

D 都不是，因為有人工步驟

Q2 為什麼建議每天合併而不是兩週合併一次？

A 工具規定

B 衝突範圍小，當天就能解

C 這樣 commit 數比較多

D 測試會跑比較快

Q3 **pipeline** 的「掃描」階段在做什麼？

A 掃描硬碟空間

B 自動檢查安全漏洞與授權問題

C 掃描使用者行為

D 把程式碼掃描成圖片

解答：CI、CD、CD

Q1 **B** 持續交付

到正式環境門口為止，有人核准。C 是連那一下都自動；D 誤以為有人工就不算。

Q2 **B** 衝突範圍小，當天就能解

痛苦來自累積，不是來自整合本身。

Q3 **B** 自動檢查安全漏洞與授權問題

掃描是最常被砍掉的一格，砍掉的代價在三個月後才出現。

一句話帶走：擋在今天，不要擋在兩週後。

Git 的四層，懂了就不用背指令

要回答的問題：為什麼 Git 指令那麼多，老手卻說「其實只有四個地方」？

- 一、 檔案的三種狀態
- 二、 四層與資料流向
- 三、 三個最常見的誤會

檔案的三種狀態

`git status` 每一行都在講這件事



還有第四種 未追蹤 (untracked, Git 根本沒在管的新檔案)；第一次 clone 下來時，所有檔案都是已追蹤且未修改。

在 **Git** 眼裡不是「有沒有存檔」，是「有沒有被記進快照」。

四層與資料流向

Pro Git 講三個區域；課網多加第四層遠端，因為協作時東西要離開你的電腦



原文對照：Pro Git 兩句話

這兩句是本課的心智模型來源

“ **Git** 會把你的檔案標記為三種主要的狀態：已提交 (**committed**)、已修改 (**modified**) 及已預存 (**staged**)。

—— Pro Git 繁中版 p.18

“ **git fetch** 命令只會下載資料到你的版本庫，它並不會自動合併你的任何工作內容，也不會自動修改你正在修改的東西。

—— Pro Git 繁中版 p.49

fetch 是「先看看別人做了什麼」，**merge** 或 **pull** 才是「把它併進來」。

三個最常見的誤會

分散式版本控制 (每個人手上都有完整歷史, 不是只有伺服器有)

✗ 大家通常以為

commit 之後東西就上雲了

✓ 其實



commit 只到本地版本庫; 沒 **push** 之前, 沒有任何東西離開你的電腦

✗ 大家通常以為

pull 跟 **fetch** 一樣

✓ 其實



fetch 只下載資料到你的版本庫, 不會自動合併; **pull** 是 **fetch** 加上自動合併

✗ 大家通常以為

遠端是「正本」, 本機是「副本」

✓ 其實



每一份 **clone** 都是完整歷史; **GitHub** 只是大家約好的集合點, 所以沒網路也能 **commit**

指令只是在搬東西, 搬對方向就不用背。

隨堂測驗：Git 的四層

Q1 你改了一個檔案並 **git add**，然後 **git commit**。這個改動現在記到哪一層了？

A 只在工作目錄

B 預存區

C 本地版本庫，還沒到遠端

D 遠端

Q2 **git fetch** 之後，你的工作目錄會改變嗎？

A 會，自動更新成遠端最新

B 不會，**fetch** 只下載，不合併

C 會，但只有新檔案

D 看遠端有沒有設定

Q3 網路斷了，你還能 **commit** 嗎？

A 不能，**commit** 要連 GitHub

B 能，**commit** 只到本地版本庫

C 能，但 **commit** 會排隊等網路

D 只能 **add** 不能 **commit**

解答: Git 的四層

Q1 **C** 本地版本庫，還沒到遠端
`commit` 只到本地版本庫，`push` 才到遠端。

Q2 **B** 不會，`fetch` 只下載，不合併
Pro Git p.49 原文：`fetch` 不會自動合併你的任何工作內容。

Q3 **B** 能，`commit` 只到本地版本庫
分散式的意思是：完整歷史在你電腦裡。

一句話帶走：指令只是在搬東西，搬對方向就不用背。

十個指令各解決什麼問題

要回答的問題：`reset` 跟 `revert` 都是「復原」，為什麼一個安全一個危險？

- 一、 十個指令，一張表
- 二、 **`reset`** 與 **`revert`**：一個改歷史，一個加歷史
- 三、 **`merge`**、**`rebase`**、**`stash`** 先看一眼

十個指令，一張表

每個指令都在回答同一個問題：把什麼，從哪一層，搬到哪一層

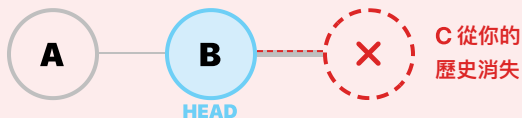
指令	解決什麼問題	搬哪兩層
add	這個改動下次要記	工作目錄 → 預存區
commit	把預存區拍成一張快照	預存區 → 本地版本庫
push	把本地的提交送到遠端	本地版本庫 → 遠端
fetch	把遠端的新東西抓下來，先不動我的	遠端 → 本地版本庫
pull	fetch 之後順便合併進目前分支	遠端 → 工作目錄
merge	把另一條分支的工作併進來	本地版本庫內
rebase	把我的提交搬到另一條分支的尾巴重放	本地版本庫內
reset	把分支指標退回某個提交	本地版本庫、預存區、可選工作目錄
revert	做一個新提交，內容是把某個提交反過來	本地版本庫內，多一個提交
stash	手上的改動先收起來，去做別的事	工作目錄與預存區 → 一個暫存堆

十個裡只有 **push**、**fetch**、**pull** 會碰到遠端，其他七個都在你的電腦裡。

reset 與 revert：一個改歷史，一個加歷史

同樣是「復原」，差別在你動的是誰的歷史

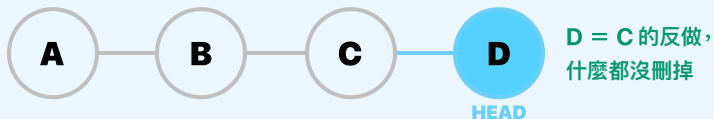
`git reset` 把分支指標退回 B，C 從歷史消失



使用時機

只在**自己電腦裡**、還沒 **push** 的提交

`git revert` 多一個 D，把 C 反過來做，A 到 C 都還在



使用時機

已經 **push**、**別人已經拿到的** 共用歷史

共用的歷史用 **revert**，只在自己電腦裡的用 **reset**。

merge、rebase、stash 先看一眼

衝突與取捨在第三單元詳講；這裡先建立分類

合併家族

要合併的分支直接超前目前分支

快進 (fast-forward, 沒有分岔, 指標直接往前推)

兩邊都有新提交

三方合併 (three-way merge, 用兩端快照加共同祖先算出結果), 產生一個有兩個親代的合併提交

想要歷史是一直線

rebase 把你的提交搬到對方尾巴重放, 之後就能快進

先收起來

git stash 改動收進一個堆, 工作目錄變乾淨

git stash apply 剛才的改動放回來

不是提交, 也不會離開你的電腦

PRO GIT p.246

this flag (`--hard`) is the only way to make the reset command dangerous, and one of the very few cases where Git will actually destroy data.

PRO GIT p.91

Do not rebase commits that exist outside your repository.

只在自己電腦裡的可以改歷史, 出了門的只能加歷史。

隨堂測驗：十個指令

Q1 你昨天 **push** 了一個提交，今天發現它壞了，同事已經 **pull** 過。怎麼復原？

A `git reset --hard` 到前一個提交再 **push**

B `git revert` 那個提交再 **push**

C 直接刪掉遠端分支重建

D `git stash`

Q2 **git reset** 什麼時候會真的弄丟資料？

A 任何時候

B 只有加 `--hard`

C 只有在遠端

D 從來不會

Q3 你手上改到一半，主管要你先修一個緊急 **bug**。先用哪個指令？

A `commit -m "wip"`

B `git stash`

C `git reset --hard`

D `git revert`

想 30 秒，翻下一頁對答案。

解答：十個指令

Q1 **B** **git revert 再 push**

歷史已經共用，只能加歷史。A 會讓同事手上的歷史對不起來，而且需要 force push。

Q2 **B** 只有加 **--hard** 的時候

Pro Git p.246: **--hard** 是唯一讓 reset 危險的旗標。

Q3 **B** **git stash**

A 也可以，但會留一個半成品提交；C 會把改動弄丟。

一句話帶走：只在自己電腦裡的可以改歷史，出了門的只能加歷史。

Agenda

- 01 文化與自動化 ✓
- 02 Git 的原理與指令 ✓
- 03 **GitHub 協作與規矩**
- 04 AI 時代的責任與實作

SO FAR

工具會用了，
但團隊還不會合作

NEXT

把「我改了什麼、
為什麼」寫成別人
與機器都讀得懂

從 **commit** 訊息到 **Pull Request**

要回答的問題：為什麼 **commit** 訊息要照格式寫？機器真的會讀嗎？

- 一、 **Conventional Commits**：訊息給機器讀
- 二、 **Pull Request**：討論在合併之前
- 三、 **Issue** 與看板：先有票，再有分支

Conventional Commits: 訊息給機器讀

Conventional Commits (慣例式提交, commit 訊息的輕量規範) · **SemVer** (語義化版本, x.y.z 三段分別是不相容改動、新功能、修錯)

結構	例子	對應到 SemVer
type: 描述	fix: 修正登入逾時沒有清除 session	PATCH (z)
type(範圍): 描述	feat(api): 新增匯出報表端點	MINOR (y)
內文或頁尾寫 BREAKING CHANGE	feat!: 移除 v1 登入端點	MAJOR (x)
其他常用 type	docs 文件 · refactor 重構行為不變 · chore 雜務 · test 測試	不動版號

機器讀來做什麼

掃一遍提交歷史 → 有 feat 升 y、只有 fix 升 z、有 BREAKING CHANGE 升 x, 順便把描述整理成變更記錄

大家通常會想

「fix bug」「update」就好, 反正 diff 都看得到 → 但 diff 只講改了什麼, 訊息才講為什麼

Pull Request: 提議合併, 討論在合併之前

Pull Request (PR, 提議把某條分支的改動合併進專案, 並在合併前讓別人討論與審查)



PR 的價值不在按鈕, 在**合併前那段時間**。

一個合格的 **PR**: 四段描述, 三種角色

diff (兩個版本之間的差異清單, PR 頁面的 **Files changed** 分頁就是它)

描述四段

1

做了什麼

一句話講改動的內容, 不是列檔名

2

為什麼

連到哪一個 **Issue**、解決誰的問題

3

怎麼測

寫得出 **可以重跑的步驟或指令**, 不是「跑了測試」

4

風險或還沒做的

自己先講出來, 審查的人就不用猜

三種角色

提 **PR** 的人

寫描述: 做了什麼、為什麼、怎麼測、風險在哪

審查的人

讀 **diff**、留言、要求修改或核准。這件事叫 **Code Review**

機器

跑測試與掃描, 把結果貼在 **PR** 上

跟四段提問格式同一個精神: **把對方要問的先寫出來。**

Issue 與看板：先有票，再有分支

Issue (議題，用來記錄想法、回饋、待辦或錯誤的一則紀錄；一個 Issue 一件事)

STEP 1

開 Issue

討論要不要做、做什麼。在這裡吵完，不要在 PR 裡吵



STEP 2

分支從它長出來

看板把 Issue 排成待辦、進行中、完成三欄，一眼看到誰在做什麼



STEP 3

PR 回頭關掉它

在 PR 描述寫 `Closes #編號`，合併進預設分支時自動關閉



這個順序的好處只有一個，但很大：每個改動都查得到「當初為什麼要做」。沒有 Issue 的分支，三個月後就是一堆無主的改動。

訊息與描述是寫給三個月後的人看的，包括三個月後的你。

隨堂測驗：commit 訊息到 PR

Q1 提交訊息 feat!: 移除舊的匯出 API 會讓自動版號工具把哪一段加一？

A x (MAJOR)

B y (MINOR)

C z (PATCH)

D 不會動版號

Q2 下列哪一個描述最符合 PR 的定位？

A 把程式碼備份到 GitHub

B 提議合併，並在合併前討論與審查

C 合併後通知大家

D 開 Issue 的另一種方式

Q3 一個 PR 描述只寫「修好了」。缺了四段裡的哪幾段？

A 只缺「怎麼測」

B 只缺「為什麼」

C 四段全缺

D 什麼都不缺，diff 會說話

想 30 秒，翻下一頁對答案。

解答: **commit** 訊息到 **PR**

Q1 **A** **x (MAJOR)**

驚嘆號或 BREAKING CHANGE 頁尾代表不相容改動。

Q2 **B** 提議合併, 並在合併前討論與審查

關鍵字是「提議」與「合併前」。

Q3 **C** 四段全缺

D 是最常見的想法, 但 **diff** 只講改了什麼。

一句話帶走: 訊息與描述是寫給三個月後的人看的。

BRANCH PROTECTION

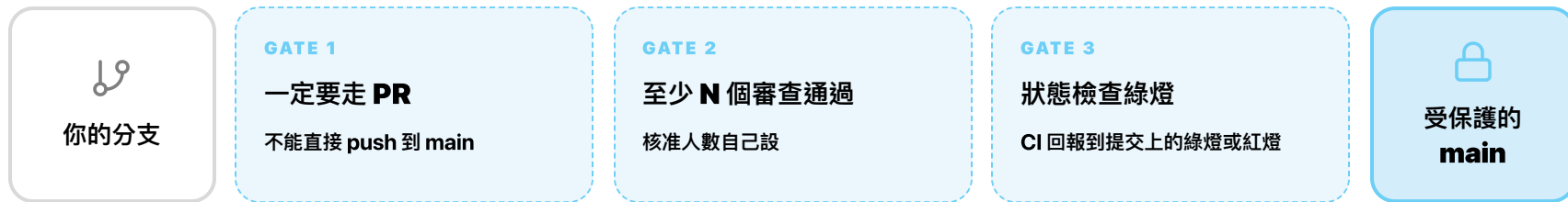
把規矩寫進工具：分支保護與必要檢查

要回答的問題：為什麼規矩寫在 wiki 沒人守，寫進 GitHub 就守了？

- 一、 三道閘門
- 二、 保護規則能設什麼、**Rulesets** 是什麼
- 三、 為什麼是「寫進工具」而不是「寫進文件」

三道閘門：任何一道沒過，按鈕就是灰的

branch protection rule (分支保護規則，對某條分支設定「要滿足什麼才能推或合併」)



預設就有

規則一建立，預設擋 **force push**、擋刪除該分支。第 4 課那句「共用的歷史不能改」就是靠這條守住

大家通常會想

擋管理員太麻煩，自己人不用 → 但出事最多的往往是有權限的人趕時間繞過流程

保護規則能設什麼

狀態檢查 (status check, CI 把測試或掃描結果回報到提交上的綠燈或紅燈)

設定	效果	文件裡的細節
Require a pull request before merging	不能直接 push, 一定要開 PR	可以要求 N 個核准
Dismiss stale approvals	有新提交進來, 舊的核准作廢	避免「核准後又偷改」
Require status checks	狀態檢查要過才能合併	可指定只接受某個 App 回報
預設行為	擋 force push 、擋刪除該分支	規則一建立就生效
Include administrators	管理員也要守	預設管理員可以跳過

RULESETS 是什麼

ruleset (規則集, GitHub 較新的一套保護機制, 功能涵蓋分支保護規則); 跟舊規則可以並存, 所有適用的規則都會生效

差在哪

舊規則一條分支只套一條, **ruleset** 可多條同時生效; 除了分支還能管標籤與 **commit** 訊息格式, 所以 **Conventional Commits** 也能強制

為什麼是「寫進工具」而不是「寫進文件」

✗ 寫在文件

✓ 寫進工具

新人要先讀完才知道



第一次違規就被擋，當場學會

趕時間時第一個被跳過



趕時間也跳不過

出事後才發現沒守



出事前就擋住

自動化的不只是建置與部署，還有規矩本身。

隨堂測驗：分支保護與必要檢查

Q1 你在 **PR** 拿到一個核准後又推了新提交，核准不見了。哪個設定造成的？

A Require status checks

B Dismiss stale approvals

C Include administrators

D Rulesets 的暫停

Q2 分支保護規則一建立，預設會擋什麼？

A 只擋刪分支

B 只擋 force push

C force push 與刪分支都擋

D 什麼都不擋，要逐項打開

Q3 為什麼課網說這些規則是「寫進工具而不是寫進文件」？

A 文件太長沒人看

B 違規當場被擋，不靠自律

C GitHub 不支援文件

D 文件會過期

解答：分支保護與必要檢查

Q1 **B** **Dismiss stale approvals**
新提交讓舊核准作廢，避免核准後偷改。

Q2 **C** **force push** 與刪分支都擋
GitHub 文件明寫兩者預設都停用。

Q3 **B** 違規當場被擋，不靠自律
A 與 D 是文件的缺點，但不是課綱給的理由。

一句話帶走：規矩寫進工具，合併按鈕自己會變灰。

AI 寫的程式碼，誰負責

要回答的問題：按下 **approve** 的那個人，知道自己在為什麼負責嗎？

- 一、 三個新問題：版權、審查責任、機密
- 二、 每個問題你今天就能做的
- 三、 這一節沒有標準答案，規範還在形成中

三個新問題，三個被打破的假設

虛線是因為三個都還沒有定論；第三欄是我的整理

一段 AI 產出的程式碼，進了你的 **commit**



版權與授權

模型學過的程式碼授權是什麼？它吐給你的這段算誰的？你放進專案用什麼授權？

原本的假設：程式碼是人寫的，或來自有授權聲明的套件



審查責任

提 PR 的人沒讀懂 AI 寫的，審查的人也沒讀懂，誰按 **approve** 誰負責？

原本的假設：寫的人至少讀過自己寫的



機密外洩

貼進 AI 工具的東西去了哪？金鑰、客戶資料、內部邏輯有沒有被拿去訓練或留存？

原本的假設：程式碼只在公司的機器上

授權 (license)，程式碼作者允許別人怎麼用的條款，例如 MIT、GPL)

機密 (secret)，不能離開組織的東西：金鑰、憑證、個資、未公開的商業邏輯)

每個問題，你今天就能做的

第三欄是講師的建議做法，不是業界共識；第二欄的狀態 2026 年 9 月仍在變

問題	沒有定論的部分	你今天就能做的
版權與授權	模型輸出到底受不受著作權保護，各國還在判	專案裡標明哪些段落是 AI 生成；用工具檢查產出有沒有跟公開程式碼大段相同
審查責任	「AI 寫的」能不能當免責理由	看不懂的程式碼不進 repo；審查時要求作者解釋每一段為什麼
機密外洩	各家工具的資料留存政策不同且會改	金鑰與客戶資料永遠不貼；用公司核准的工具；貼之前把識別資訊拿掉



流程裡有一個位置逃不掉：按 **approve** 的人。第 6 課把審查設成必要條件，代表組織把責任放在那個按鈕上。

AI 讓寫的人變快，沒有讓按的人變輕。

隨堂測驗：AI 寫的程序碼誰負責

Q1 下列哪一個做法最能降低「機密外洩」的風險？

A 只用免費的 AI 工具

B 貼之前拿掉金鑰與個資，只用核准工具

C 貼完之後刪除對話

D 只貼錯誤訊息，錯誤訊息一定安全

Q2 「這段是 AI 寫的」在審查時可以當免責理由嗎？

A 可以，責任在 AI 廠商

B 不行，按 approve 的人仍要負責

C 只有開源專案不行

D 看模型大小

Q3 為什麼課網這一節不給標準答案？

A 講師不知道

B 規範還在形成中，先讓學員意識到問題

C 這三個問題不重要

D 答案在下一個單元

解答: AI 寫的程式碼誰負責

- Q1 **B** 拿掉金鑰與個資, 只用核准工具
C 刪對話不等於對方沒留存; D 錯誤訊息裡常帶路徑、帳號、金鑰片段。

- Q2 **B** 不行, 按 **approve** 的人仍要負責
這是本課立場: 流程把責任放在那個按鈕上。

- Q3 **B** 規範還在形成中
課網原話: 先讓學員意識到問題在哪。

一句話帶走: AI 讓寫的人變快, 沒有讓按的人變輕。

實作：整理提交歷史，開一個合格的 **PR**

要回答的問題：不寫一行程式，能不能把一段亂七八糟的提交歷史變成別人看得懂的 **PR**？

- 一、 五個步驟：從 **messy** 到 **tidy**
- 二、 互動式 **rebase** 怎麼操作
- 三、 審查清單七格：分數大半在這裡

跟著做：從 **messy** 到一個合格的 **PR**

1

拿到亂的歷史

```
$ git clone <你 fork 的 repo> && git checkout messy && git log --oneline
```

fork 講師範本或用 Use this template 建一份。main 只有初始提交，messy 上有 6 個隨手寫的 commit。這份 repo 是你自己的，messy 沒有別人在用，所以可以改寫

2

整理成 **3** 到 **4** 個，訊息改成規範

```
$ git rebase -i main
```

驗收：每則訊息有正確的 type，改動相關的合併成一則。操作方式見下一頁

3

推到新分支 **tidy**，開 **PR** 寫四段描述

```
$ git push -u origin HEAD:tidy
```

不要推回 messy，這樣就**不用 force push**。PR 從 tidy 到 main，描述寫做了什麼、為什麼、怎麼測、風險

4

讓合併按鈕被擋一次

Settings → Rules → Rulesets → New: Target **main**、Require a PR、核准數 1、Bypass 留空、Active

截下被擋的畫面與原因文字。一個人的 repo 無法核准自己的 PR，這正好證明規則生效，不要求合併成功

5

自審，再讓 **AI** 扮 **Tech Lead** 質疑

提示：扮演資深 Tech Lead，針對測試覆蓋、改動範圍與邊界情境逐條質疑我這個 PR

先自己依七格清單審一次，再把質疑與你的回應寫進 PR 留言，**至少三則**

互動式 rebase 怎麼操作

互動式 **rebase** (`git rebase -i`, 讓你逐個改寫、合併、重排還沒被別人拿去的提交)

git-rebase-todo

```
$ git rebase -i main
pick    a1b2c3d fix
squash  e4f5g6h update
squash  i7j8k9l 再改一次
reword  m0n1o2p asdf
pick    q3r4s5t 補文件
squash  u6v7w8x 再補一點文件
# 上面的先套用；存檔關閉後開始執行
```

`git rebase -i main`

打開編輯器，列出 **messy** 上從 **main** 之後的 6 行，每行開頭是 **pick**

pick → **squash (s)**

要併進上一則。存檔關閉後 **Git** 再開一次編輯器，讓你寫合併後的訊息

pick → **reword (r)**

只改訊息。存檔關閉後逐則讓你改

直接搬動行

順序不對就上下移動；上面的先套用

`git rebase --abort`

弄亂了就回到開始前，什麼都沒發生。可以放心試

改寫還沒被別人拿去的歷史是安全的，這是那條鐵則的反面。

審查清單七格

逐格打勾，或寫「沒做到，因為……」。前四格來自前六課，後三格是第 7 課加的

格	檢查什麼	來自哪一課
訊息	每則都有 type ，描述寫「做了什麼」不是「改了哪個檔」	commit 到 PR
粒度	一則提交一件事，相關改動合在一起	CI/CD 小批量
描述	四段齊全，「怎麼測」寫得出具體步驟	commit 到 PR
閘門	PR 有被規則擋過的痕跡（沒核准前合併按鈕是灰的）	分支保護
版權	有沒有標明哪些內容是 AI 生成	AI 誰負責
責任	每一段改動你都講得出為什麼	AI 誰負責
機密	diff 裡沒有金鑰、憑證、個資	AI 誰負責

PR 是寫給審查者看的，包括三個月後的你。

隨堂測驗：實作

Q1 你要把還沒 **push** 的 **6** 個提交合併成 **3** 個，用什麼？

A git merge

B git rebase -i

C git revert

D git stash

Q2 為什麼整理歷史要在 **push** 之前做？

A push 之後技術上做不到

B 歷史共用了，改寫會讓別人對不起來

C GitHub 會拒絕整理過的歷史

D 沒有為什麼，習慣而已

想 30 秒，翻下一頁對答案。

解答：實作

Q1 **B** **git rebase -i**

互動式 rebase 可以合併、改寫、重排還沒 push 的提交。

Q2 **B** 歷史共用了，改寫會讓別人對不起來

A 是誤解：技術上做得到，但違反「不要 rebase 存在於你版本庫之外的提交」。

一句話帶走：**PR** 是寫給審查者看的，包括三個月後的你。

TAKEAWAY

八句話帶走

CULTURE

DevOps 不解決工具問題，它解決人的問題

CI / CD

擋在**今天**，不要擋在兩週後

FOUR LAYERS

指令只是在搬東西，**搬對方向**就不用背

RESET / REVERT

出了門的歷史**只能加**，不能改

COMMIT / PR

訊息與描述是寫給**三個月後的人**看的

PROTECTION

規矩寫進工具，合併按鈕**自己**會變灰

AI & APPROVE

AI 讓寫的人變快，**沒有讓按的人**變輕

THE LAB

PR 是寫給**審查者**看的

如果只記得一句話：**工具固定流程，文化決定流程**

八份個人作業

01 用 CALMS 看你的團隊

五面向各寫「做到 / 沒做到」，挑一格寫下週就能做的小改變；AI 扮 DevOps 顧問，只問三個問題

02 畫你的 pipeline

四階段，每階段寫一個「失敗就擋」的具體條件；AI 扮 SRE 指出你漏了哪一個

03 把 status 對到四層

改檔 → status → add → status → commit → log → fetch，每步標「記在哪一層」；AI 畫 ASCII 四層圖，你驗證對錯

04 六個情境，六個指令

每題寫指令與一句理由；挑一題讓 AI 拆解復原步驟，你逐步在自己 repo 驗證

05 三則訊息改成規範

原句、我改的、AI 改的、差異一句話；請 AI 解釋規範怎麼降低 CI/CD 追蹤成本

06 讓合併按鈕變灰

對 main 建 ruleset，截 PR 被擋與直接 push 被拒；寫一段「早三個月哪次事故不會發生」

07 AI 程式碼提交守則

三條規則各對一個問題，每條一句理由；AI 找出這三條在什麼情況會失效，你回應

08 就是那個實作

繳：整理前後 git log --oneline 各一份、PR 連結、七格清單、回應 AI 質疑的留言截圖



作業 06 注意：免費方案的 ruleset 只對公開 repo 生效；私人 repo 改用 branch protection rule，設定項目一樣

八份共同結構：你先自己做 → 貼給 AI 要它扮某個角色質疑你 → 把質疑與你的回應一起繳。不是要它幫你做

Reference

- 1 Chacon, S. & Straub, B. *Pro Git* 繁體中文版。→ 今天四層、三種狀態、reset 三棵樹、Rewriting History 的來源；建議讀第 1 到第 3 章 (p.12–p.91、p.228、p.234–p.246)
- 2 Atlassian。CALMS Framework / What is DevOps。→ CALMS 五個面向的名稱與「DevOps 解決人的問題」那句話；十分鐘讀完
- 3 AWS。What is Continuous Integration? / What is Continuous Delivery? → CI、持續交付、持續部署的定義與「有沒有人工核准」那條界線
- 4 Conventional Commits 1.0.0 繁體中文版。→ type 與 SemVer 的對應關係、其他常用 type 清單；五分鐘讀完規範本文
- 5 GitHub Docs。About pull requests / About issues / About protected branches / About rulesets。→ 作業 06 的操作說明就在這裡
- 6 git-scm.com。git-revert 官方手冊。→ revert 的定義；Pro Git 繁中版沒有獨立章節，出處用官方手冊

所有網路資料 2026-09-13 查閱。每頁左下角都標了出處與頁碼，查原文對得到位置。